



Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Agentic AI (IA462), Semester VIII

Instructions

1. Save your file as LAB_No_Roll_No.doc (e.g. LAB_6_U21AI001.doc).
2. Paste a snapshot of each step followed along with the result(s).
3. Record your observations and comments after the results.

Experiment 6

Role-Based LLM Agent Design with Structured Message Passing

Aim

To specify LLM agents as explicit roles, each with a goal, permitted tools, permitted recipients and a required output schema, coordinate them through a manager using structured messages with a reviewer that sends faulty work back to the responsible role, and compare the team against a single generalist agent by task success, model calls and role violations.

Theory

A single agent given every tool and one long prompt that mixes retrieval, writing and checking must decide at each step which of those jobs it is doing, and nothing prevents it from skipping one. Role-based design splits the work into roles, each defined by a goal, a system prompt, the tools it may call, the roles it may message and the fields its output must contain. A manager decomposes the task and delegates; workers return structured messages; a reviewer checks the draft against the sources and either approves it or sends it back to the role responsible for the fault.

A specification is only as strong as its enforcement. A permission written in a prompt is a request the model may ignore; the same permission checked by the orchestrator before a tool call runs or a message is delivered is a rule. Here `validate()` is that check, and every rejection is logged as a role violation. As in Experiments 3 and 5 the model side is scripted: each role is a function that does its job but commits named faults at stated rates, and a real LLM changes only those functions. The team's price is extra model calls, measured alongside task success.

Table 1. The four roles to be implemented.

Role	Responsibility	Permitted tools	May send to	Required output fields
Manager	decomposes the task, delegates, routes revisions, delivers the final answer	none	Researcher, Writer, Reviewer, User	subtasks
Researcher	finds both entities, reads their values, computes the result	<code>search()</code> , <code>read_doc()</code> , <code>calculator()</code>	Manager	values, sources, result
Writer	turns the finding into a cited answer of at most 60 words	none	Manager	draft, citations
Reviewer	re-reads the cited sources and checks values, winner and citations	<code>read_doc()</code>	Manager	verdict, reasons

Algorithm

1. Build a document store of at least eight short notes (model name, benchmark, accuracy, parameter count), two of whose titles nearly duplicate others; the tools `search()`, `read_doc()` and `calculator()`; and eight comparison tasks, each scored on a checklist: correct number, correct winner, both correct sources cited, at most 60 words. Write a role specification for each role in Table 1 and implement `validate()`, which rejects any tool call outside the role's

permitted tools, any message to a role it may not send to and any output missing a required field, and logs each rejection as a role violation.

2. Implement the orchestrator: the Manager assigns the task to the Researcher, passes the finding to the Writer and the draft to the Reviewer; on a revise verdict it routes the task back to the role named in the reasons, for at most two revision rounds, then delivers the final draft to the user. Run the same tasks with a single generalist agent that holds all three tools and one merged prompt. For both systems record task success, model calls per role, role violations and revisions, and print the full message trace for every task.

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the program below, including the four role specifications, the message format, one complete message trace that contains a revision, and the comparison of the team and the generalist by task success, model calls, role violations and revisions, reported over at least ten seeds.

Observations

Comment on which tasks the team completes that the generalist fails and which role caught the fault in each, on the model calls spent per additional correct answer, on why the generalist records no role violations and whether that makes it safer, on which role produces the most violations and whether the fix belongs in its prompt or in its permissions, and on what happens to a fault the Reviewer misses.

Exercises

1. Ablate the team in two ways: remove the Reviewer, and merge the Researcher and Writer into a single role that holds the Researcher's tools. Report task success, model calls and role violations for the full team, both variants and the generalist over the same seeds, and state which role contributes the most correct answers per model call it consumes.

2. Write five injected instructions that push a role outside its specification: the Writer told to search, a document telling the Researcher to send its finding straight to the user, the Reviewer told to fix the draft itself, a document telling the Researcher to report a false value, and an assignment telling the Researcher to omit its sources. Run each case with the permissions stated only in the prompts and again with `validate()` enforcing them, and report for each case how many attempts were blocked, how many took effect, and which case `validate()` cannot stop.